

第29章 LCD—液晶显示中英文

本章参考资料：《STM32F4xx-中文参考手册》、《STM32F4xx 数据手册》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

在前面我们学习了如何使用 FSMC 外设控制液晶屏并用它显示各种图形，本章讲解如何控制液晶屏显示文字。使用液晶屏显示文字时，涉及到字符编码与字模的知识。

29.1 字符编码

由于计算机只能识别 0 和 1，文字也只能以 0 和 1 的形式在计算机里存储，所以我们需要对文字进行编码才能让计算机处理，编码的过程就是规定特定的 01 数字串来表示特定的文字，最简单的字符编码例子是 ASCII 码。

29.1.1 ASCII 编码

学习 C 语言时，我们知道在程序设计中使用 ASCII 编码表约定了一些控制字符、英文及数字。它们在存储器中，本质也是二进制数，只是我们约定这些二进制数可以表示某些特殊意义，如以 ASCII 编码解释数字“0x41”时，它表示英文字符“A”。ASCII 码表分为两部分，第一部分是控制字符或通讯专用字符，它们的数字编码从 0~31，见表 28-8，它们并没有特定的图形显示，但会根据不同的应用程序，而对文本显示有不同的影响。ASCII 码的第二部分包括空格、阿拉伯数字、标点符号、大小写英文字母以及“DEL（删除控制）”，这部分符号的数字编码从 32~127，除最后一个 DEL 符号外，都能以图形的方式来表示，它们属于传统文字书写系统的一部分。

表 28-8 ASCII 码中的控制字符或通讯专用字符

十进制	十六进制	缩写/字符	解释
0	0	NUL(null)	空字符
1	1	SOH(start of headline)	标题开始
2	2	STX (start of text)	正文开始
3	3	ETX (end of text)	正文结束
4	4	EOT (end of transmission)	传输结束
5	5	ENQ (enquiry)	请求
6	6	ACK (acknowledge)	收到通知
7	7	BEL (bell)	响铃
8	8	BS (backspace)	退格
9	9	HT (horizontal tab)	水平制表符
10	0A	LF (NL line feed, new line)	换行键
11	0B	VT (vertical tab)	垂直制表符
12	0C	FF (NP form feed, new page)	换页键
13	0D	CR (carriage return)	回车键
14	0E	SO (shift out)	不用切换
15	0F	SI (shift in)	启用切换

16	10	DLE (data link escape)	数据链路转义
17	11	DC1 (device control 1)	设备控制 1
18	12	DC2 (device control 2)	设备控制 2
19	13	DC3 (device control 3)	设备控制 3
20	14	DC4 (device control 4)	设备控制 4
21	15	NAK (negative acknowledge)	拒绝接收
22	16	SYN (synchronous idle)	同步空闲
23	17	ETB (end of trans. block)	传输块结束
24	18	CAN (cancel)	取消
25	19	EM (end of medium)	介质中断
26	1A	SUB (substitute)	替补
27	1B	ESC (escape)	换码(溢出)
28	1C	FS (file separator)	文件分割符
29	1D	GS (group separator)	分组符
30	1E	RS (record separator)	记录分离符
31	1F	US (unit separator)	单元分隔符

表 28-9 ASCII 码中的字符及数字

十进制	十六进制	缩写/字符	十进制	十六进制	缩写/字符
32	20	(space)空格	80	50	P
33	21	!	81	51	Q
34	22	"	82	52	R
35	23	#	83	53	S
36	24	\$	84	54	T
37	25	%	85	55	U
38	26	&	86	56	V
39	27	'	87	57	W
40	28	(	88	58	X
41	29	)	89	59	Y
42	2A	*	90	5A	Z
43	2B	+	91	5B	[
44	2C	,	92	5C	\
45	2D	-	93	5D	]
46	2E	.	94	5E	^
47	2F	/	95	5F	_
48	30	0	96	60	`
49	31	1	97	61	a
50	32	2	98	62	b
51	33	3	99	63	c
52	34	4	100	64	d
53	35	5	101	65	e
54	36	6	102	66	f
55	37	7	103	67	g
56	38	8	104	68	h
57	39	9	105	69	i
58	3A	:	106	6A	j
59	3B	;	107	6B	k

60	3C	<	108	6C	l
61	3D	=	109	6D	m
62	3E	>	110	6E	n
63	3F	?	111	6F	o
64	40	@	112	70	p
65	41	A	113	71	q
66	42	B	114	72	r
67	43	C	115	73	s
68	44	D	116	74	t
69	45	E	117	75	u
70	46	F	118	76	v
71	47	G	119	77	w
72	48	H	120	78	x
73	49	I	121	79	y
74	4A	J	122	7A	z
75	4B	K	123	7B	{
76	4C	L	124	7C	
77	4D	M	125	7D	}
78	4E	N	126	7E	~
79	4F	O	127	7F	DEL (delete) 删除

后来，计算机引进到其它国家的时候，由于他们使用的不是英语，他们使用的字母在 ASCII 码表中没有定义，所以他们采用 127 号之后的位来表示这些新的字母，还加入了各种形状，一直编号到 255。从 128 到 255 这些字符被称为 ASCII 扩展字符集。至此基本存储单位 Byte(char)能表示的编号都被用完了。

29.1.2 中文编码

由于英文书写系统都是由 26 个基本字母组成，利用 26 个字母组可合出不同的单词，所以用 ASCII 码表就能表达整个英文书写系统。而中文书写系统中的汉字是独立的方块，若参考单词拆解成字母的表示方式，汉字可以拆解成部首、笔画来表示，但这样会非常复杂(可参考五笔输入法编码)，所以中文编码直接对方块字进行编码，一个汉字使用一个号码。

由于汉字非常多，常用字就有 6000 多个，如果像 ASCII 编码表那样只使用 1 个字节最多只能表示 256 个汉字，所以我们使用 2 个字节来编码。

1. GB2312 标准

我们首先定义的是 GB2312 标准。它把 ASCII 码表 127 号之后的扩展字符集直接取消掉，并规定小于 127 的编码按原来 ASCII 标准解释字符。当 2 个大于 127 的字符连在一起时，就表示 1 个汉字，第 1 个字节使用 (0xA1-0xFE) 编码，第 2 个字节使用(0xA1-0xFE)编码，这样的编码组合起来可以表示了 7000 多个符号，其中包含 6763 个汉字。在这些编码里，我们还把数学符号、罗马字母、日文假名等都编进表中，就连原来在 ASCII 里原本就

有的数字、标点以及字母也重新编了 2 个字节长的编码，这就是平时在输入法里可切换的“全角”字符，而标准的 ASCII 码表中 127 号以下的就被称为“半角”字符。

表 28-10 说明了 GB2312 是如何兼容 ASCII 码的，当我们设定系统使用 GB2312 标准的时候，它遇到一个字符串时，会按字节检测字符值的大小，若遇到连续两个字节的数值都大于 127 时就把这两个连续的字节合在一起，用 GB2312 解码，若遇到的数值小于 127，就直接用 ASCII 把它解码。

表 28-10 GB2312 兼容 ASCII 码的原理

第 1 字节	第 2 字节	表示的字符	说明
0x68	0x69	(hi)	两个字节的值都小于 127(0x7F)，使用 ASCII 解码
0xB0	0xA1	(啊)	两个字节的值都大于 127(0x7F)，使用 GB2312 解码

区位码

在 GB2312 编码的实际使用中，有时会用到区位码的概念，见图 28-34。GB2312 编码对所收录字符进行了“分区”处理，共 94 个区，每区含有 94 个位，共 8836 个码位。而区位码实际是 GB2312 编码的内部形式，它规定对收录的每个字符采用两个字节表示，第一个字节为“高字节”，对应 94 个区；第二个字节为“低字节”，对应 94 个位。所以它的区位码范围是：0101-9494。为兼容 ASCII 码，区号和位号分别加上 0xA0 偏移就得到 GB2312 编码。在区位码上加上 0xA0 偏移，可求得 GB2312 编码范围：0xA1A1—0xFEFE，其中汉字的编码范围为 0xB0A1-0xF7FE，第一字节 0xB0-0xF7（对应区号：16—87），第二个字节 0xA1-0xFE（对应位号：01—94）。

例如，“啊”字是 GB2312 编码中的第一个汉字，它位于 16 区的 01 位，所以它的区位码就是 1601，加上 0xA0 偏移，其 GB2312 编码为 0xB0A1。其中区位码为 0101 的码位表示的是“空格”符。

1区 01 0 1 2 3 4 5 6 7 8 9 0 、。·-~…“”（） 1 一~ …‘’“”（） 2 <>《》「」『』【】 3 【】±×÷∶∧∨ΣΠ 4 ∪∩≡∴√⊥∥∠（）⊙ 5 ∫∑∏∞≈≠≡≠≠≠ 6 ≡≡≡≡≡≡≡≡≡≡ 7 ℃\$%&'()*+,-./:; 8 ○●◎◇◆□■△▲※ 9 →↑↓⇄=	16区 16 0 1 2 3 4 5 6 7 8 9 0 啊阿埃挨哎唉哀皑癌 1 蔼矮岸胺案隘隘澳奥凹俺 2 按暗岸岸傲奥奥芭捌扒把 3 熬叭吧芭霸爸白扳扳邦帮 4 耙坝霸罢爸白扳扳邦帮 5 败拜拜拜班班搬搬办办办 6 版版拌拌伴伴瓣瓣办办办 7 梆榜榜榜绑绑棒棒磅磅磅 8 苞苞胞胞包包褒褒剥剥剥 9
2区 02 0 1 2 3 4 5 6 7 8 9 0 i ii iii iv v vi vii viii ix 1 x 1. 2. 3. 2 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 3 14. 15. 16. 17. 18. 19. 20. (1) (2) (3) 4 (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) 5 (14) (15) (16) (17) (18) (19) (20) ① ② ③ 6 ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ € (-) 7 (一) (二) (三) (四) (五) (六) (七) (八) (九) (十) 8 I II III IV V VI VII VIII IX 9 X XI XII	17区 17 0 1 2 3 4 5 6 7 8 9 0 薄雹爆杯堡堡饱宝抱报暴 1 豹饱钹倍狈备备备备备备备 2 贝钹狈备备备备备备备备 3 本笨崩绷彼碧弊弊弊弊弊 4 比币庇痹闭遍编编编编编 5 避陛鞭鞭边遍编编编编编 6 辨辨辨辨遍遍遍遍遍遍遍 7 别别别别彬彬斌斌斌斌斌 8 柄柄柄柄柄柄柄柄柄柄柄 9

图 28-34 GB2312 的部分区位码

2. GBK 编码

据统计，GB2312 编码中表示的 6763 个汉字已经覆盖中国大陆 99.75% 的使用率，单看这个数字已经很令人满意了，但是我们不能因为那些文字不常用就不让它进入信息时代，而且生僻字在人名、文言文中的出现频率是非常高的。为此我们在 GB2312 标准的基础上又增加了 14240 个新汉字(包括所有后面介绍的 Big5 中的所有汉字)和符号，这个方案被称为 GBK 标准。增加这么多字符，按照 GB2312 原来的格式来编码，2 个字节已经没有足够的编码，我们聪明的程序员修改了一下格式，不再要求第 2 个字节的编码值必须大于 127，只要第 1 个字节大于 127 就表示这是一个汉字的开始，这样就做到了兼容 ASCII 和 GB2312 标准。

表 28-11 说明了 GBK 是如何兼容 ASCII 和 GB2312 标准的，当我们设定系统使用 GBK 标准的时候，它按顺序遍历字符串，按字节检测字符值的大小，若遇到一个字符的值大于 127 时，就再读取它后面的一个字符，把这两个字符值合在一起，用 GBK 解码，解码完后，再读取第 3 个字符，重新开始以上过程，若该字符值小于 127，则直接用 ASCII 解码。

表 28-11 GBK 兼容 ASCII 和 GB2312 的原理

第 1 字节	第 2 字节	第 3 字节	表示的字符	说明
0x68(<7F)	0xB0(>7F)	0xA1(>7F)	(h 啊)	第 1 个字节小于 127，使用 ASCII 解码，每 2 个字节大于 127，直接使用 GBK 解码，兼容 GB2312

0xB0(>7F)	0xA1(>7F)	0x68(<7F)	(啊 h)	第 1 个字节大于 127，直接使用 GBK 码解释，第 3 个字节小于 127，使用 ASCII 解码
0xB0(>7F)	0x56(<7F)	0x68(<7F)	(癡 h)	第 1 个字节大于 127，第 2 个字节虽然小于 127，直接使用 GBK 解码，第 3 个字节小于 127，使用 ASCII 解码

3. GB18030

随着计算机技术的普及，我们后来又在 GBK 的标准上不断扩展字符，这些标准被称为 GB18030，如 GB18030-2000、GB18030-2005 等(“-”号后面的数字是制定标准时的年号)，GB18030 的编码使用 4 个字节，它利用前面标准中的第 2 个字节未使用的“0x30-0x39”编码表示扩充四字节的后缀，兼容 GBK、GB2312 及 ASCII 标准。

GB18030-2000 主要在 GBK 基础上增加了“CJK(中日韩)统一汉字扩充 A”的汉字。加上前面 GBK 的内容，GB18030-2000 一共规定了 27533 个汉字（包括部首、部件等）的编码，还有一些常用非汉字符号。

GB18030-2005 的主要特点是在 GB18030-2000 基础上增加了“CJK(中日韩)统一汉字扩充 B”的汉字。增加了 42711 个汉字和多种我国少数民族文字的编码（如藏、蒙古、傣、彝、朝鲜、维吾尔文等）。加上前面 GB18030-2000 的内容，一共收录了 70244 个汉字。

GB2312、GBK 及 GB18030 是汉字的国家标准编码，新版向下兼容旧版，各个标准简要说明见表 28-12，目前比较流行的是 GBK 编码，因为每个汉字只占用 2 个字节，而且它编码的字符已经能满足大部分的需求，但国家要求一些产品必须支持 GB18030 标准。

表 28-12 汉字国家标准

类别	编码范围	汉字编码范围	扩充汉字数	说明
GB2312	第一字节 0xA1-0xFE 第二字节 0xA1-0xFE	第一字节 0xB0-0xF7 第二字节 0xA1-0xFE	6763	除汉字外，还包括拉丁字母、希腊字母、日文平假名及片假名字母、俄语西里尔字母在内的 682 个全角字符
GBK	第一字节 0x81-0xFE 第二字节 0x40-0xFE	第一字节 0x81-0xA0 第二字节 0x40-0xFE	6080	包括部首和构件,中日韩汉字,包含了 BIG5 编码中的所有汉字,加上 GB2312 的原内容，一共有 21003 个汉字
		第一字节 0xAA-0xFE 第二字节 0x40-0xA0	8160	
GB18030-2000	第一字节 0x81-0xFE 第二字节 0x30-0x39 第三字节 0x81-0xFE 第四字节 0x30-0x39	第一字节 0x81-0x82 第二字节 0x30-0x39 第三字节 0x81-0xFE 第四字节 0x30-0x39	6530	在 GBK 基础上增加了中日韩统一汉字扩充 A 的汉字，加上 GB2312、GBK 的内容，一共有 27533 个汉字
GB18030-2005	第一字节 0x81-0xFE 第二字节 0x30-0x39 第三字节	第一字节 0x95-0x98 第二字节 0x30-0x39 第三字节 0x81-	42711	在 GB18030-2000 的基础上增加了 42711 中日韩统一汉字扩充 B 中的汉字和多种我国少数民族文字的编码（如藏、蒙古、傣、彝、朝鲜、维吾尔文等），加上前面

	0x81-0xFE 第四字节 0x30-0x39	0xFE 第四字节 0x30- 0x39		GB2312、GBK、GB18030-2000 的 内容，一共 70244 个汉字
--	--------------------------------	----------------------------	--	--

4. Big5 编码

在台湾、香港等地区，使用较多的是 Big5 编码，它的主要特点是收录了繁体字。而从 GBK 编码开始，已经把 Big5 中的所有汉字收录进编码了。即对于汉字部分，GBK 是 Big5 的超集，Big5 能表示的汉字，在 GBK 都能找到那些字相应的编码，但他们的编码是不一样的，两个标准不兼容，如 GBK 中的“啊”字编码是“0xB0A1”，而 Big5 标准中的编码为“0xB0DA”。

29.1.3 Unicode 字符集和编码

由于各个国家或地区都根据使用自己的文字系统制定标准，同一个编码在不同的标准里表示不一样的字符，各个标准互不兼容，而又没有一个标准能够囊括所有的字符，即无法用一个标准表达所有字符。国际标准化组织(ISO)为解决这一问题，它舍弃了地区性的方案，重新给全球上所有文化使用的字母和符号进行编号，对每个字符指定一个唯一的编号(ASCII 中原有的字符编号不变)，这些字符的号码从 0x000000 到 0x10FFFF，该编号集被称为 Universal Multiple-Octet Coded Character Set，简称 UCS，也被称为 Unicode。最新版的 Unicode 标准还包含了表情符号(聊天软件中的部分 emoji 表情)，可访问 Unicode 官网了解：<http://www.unicode.org>。

Unicode 字符集只是对字符进行编号，但具体怎么对每个字符进行编码，Unicode 并没指定，因此也衍生出了如下几种 unicode 编码方案(Unicode Transformation Format)。

29.1.4 UTF-32

对 Unicode 字符集编码，最自然的就是 UTF-32 方式了。编码时，它直接对 Unicode 字符集里的每个字符都用 4 字节来表示，转换方式很简单，直接将字符对应的编号数字转换为 4 字节的二进制数。如表 28-13，由于 UTF-32 把每个字符都用要 4 字节来存储，因此 UTF-32 不兼容 ASCII 编码，也就是说 ASCII 编码的文件用 UTF-32 标准来打开会成为乱码。

表 28-13 UTF-32 编码示例

字符	GBK 编码	Unicode 编号	UTF-32 编码
A	0x41	0x0000 0041	大端格式 0x0000 0041
啊	0xB0A1	0x0000 554A	大端格式 0x0000 554A

对 UTF-32 数据进行解码的时候，以 4 个字节为单位进行解析即可，根据编码可直接找到 Unicode 字符集中对应编号的字符。

UTF-32 的优点是编码简单，解码也很方便，读取编码的时候每次都直接读 4 个字节，不需要加其它的判断。它的缺点是浪费存储空间，大量常用字符的编号只需要 2 个字节就能表示。其次，在存储的时候需要指定字节顺序，是高位字节存储在前(大端格式)，还是低位字节存储在前(小端格式)。

29.1.5 UTF-16

针对 UTF-32 的缺点，人们改进出了 UTF-16 的编码方式，它采用 2 字节或 4 字节的变长编码方式(UTF-32 定长为 4 字节)。对 Unicode 字符编号在 0 到 65535 的统一用 2 个字节来表示，将每个字符的编号转换为 2 字节的二进制数，即从 0x0000 到 0xFFFF。而由于 Unicode 字符集在 0xD800-0xDBFF 这个区间是没有表示任何字符的，所以 UTF-16 就利用这段空间，对 Unicode 中编号超出 0xFFFF 的字符，利用它们的编号做某种运算与该空间建立映射关系，从而利用该空间表示 4 字节扩展，感兴趣的读者可查阅相关资料了解具体的映射过程。

表 28-14 UTF-16 编码示例

字符	GB18030 编码	Unicode 编号	UTF-16 编码
A	0x41	0x0000 0041	大端格式 0x0041
啊	0xB0A1	0x0000 554A	大端格式 0x554A
嶲	0x9735 F832	0x0002 75CC	大端格式 0xD85D DDCC

7. 注：嶲 五笔：TLHH(不支持 GB18030 码的输入法无法找到该字，感兴趣可搜索它的 Unicode 编号找到)

UTF-16 解码时，按两个字节去读取，如果这两个字节不在 0xD800 到 0xDFFF 范围内，那就是双字节编码的字符，以双字节进行解析，找到对应编号的字符。如果这两个字节在 0xD800 到 0xDFFF 之间，那它就是四字节编码的字符，以四字节进行解析，找到对应编号的字符。

UTF-16 编码的优点是相对 UTF-32 节约了存储空间，缺点是仍不兼容 ASCII 码，仍有大小端格式问题。

29.1.6 UTF-8

UTF-8 是目前 Unicode 字符集中使用得最广的编码方式，目前大部分网页文件已使用 UTF-8 编码，如使用浏览器查看百度首页源文件，可以在前几行 HTML 代码中找到如下代码：

```
1 <meta http-equiv=Content-Type content="text/html; charset=utf-8">
```

其中“charset”等号后面的“utf-8”即表示该网页字符的编码方式 UTF-8。

UTF-8 也是一种变长的编码方式，它的编码有 1、2、3、4 字节长度的方式，每个 Unicode 字符根据自己的编号范围去进行对应的编码，见表 28-15。它的编码符合以下规律：

- 对于 UTF-8 单字节的编码，该字节的第 1 位设为 0(从左边数起第 1 位，即最高位)，剩余的位用来写入字符的 Unicode 编号。即对于 Unicode 编号从 0x0000 0000-0x0000 007F 的字符，UTF-8 编码只需要 1 个字节，因为这个范围 Unicode 编号的字符与 ASCII 码完全相同，所以 UTF-8 兼容了 ASCII 码表。
- 对于 UTF-8 使用 N 个字节的编码(N>1)，第一个字节的前 N 位设为 1，第 N+1 位设为 0，后面字节的前两位都设为 10，这 N 个字节的其余空位填充该字符的 Unicode 编号，高位用 0 补足。

表 28-15 UTF-8 编码原理(x 的位置用于填充 Unicode 编号)

Unicode(16 进制)	UTF-8 (2 进制)				
编号范围	第一字节	第二字节	第三字节	第四字节	第五字节
00000000-0000007F	0xxxxxxx				

00000080-000007FF	110xxxxx	10xxxxxx			
00000800-0000FFFF	1110xxxx	10xxxxxx	10xxxxxx		
00010000-0010FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	
...	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

8. 注：实际上 utf-8 编码长度最大为四个字节，所以最多只能表示 Unicode 编码值的二进制数为 21 位的 Unicode 字符。但是已经能表示所有的 Unicode 字符，因为 Unicode 的最大码位 0x10FFFF 也只有 21 位。

UTF-8 解码的时候以字节为单位去看，如果第一个字节的 bit 位以 0 开头，那就是 ASCII 字符，以单字节进行解析。如果第一个字节的数据位以“110”开头，就按双字节进行解析，3、4 字节的解析方法类似。

UTF-8 的优点是兼容了 ASCII 码，节约空间，且没有字节顺序的问题，它直接根据第 1 个字节前面数据位中连续的 1 个数决定后面有多少个字节。不过使用 UTF-8 编码汉字平均需要 3 个字节，比 GBK 编码要多一个字节。

29.1.7 BOM

由于 UTF 系列有多种编码方式，而且对于 UTF-16 和 UTF-32 还有大小端的区分，那么计算机软件在打开文档的时候到底应该用什么编码方式去解码呢？有的人就想到在文档最前面加标记，一种标记对应一种编码方式，这些标记就叫做 BOM(Byte Order Mark)，它们位于文本文件的开头，见表 28-16。注意 BOM 是对 Unicode 的几种编码而言的，ANSI 编码没有 BOM。

表 28-16 BOM 标记

BOM 标记	表示的编码
0xEF 0xBB 0xBF	UTF-8
0xFF 0xFE	UTF-16 小端格式
0xFE 0xFF	UTF-16 大端格式
0xFF 0xFE 0x00 0x00	UTF-32 小端格式
0x00 0x00 0xFE 0xFF	UTF-32 大端格式

但由于带 BOM 的设计很多规范不兼容，不能跨平台，所以这种带 BOM 的设计没有流行起来。Linux 系统下默认不带 BOM。

29.2 什么是字模？

有了编码，我们就能在计算机中处理、存储字符了，但是如果计算机处理完字符后直接以编码的形式输出，人类将难以识别。来，在 2 秒内告诉我 ASCII 编码的“0x25”表示什么字符？不容易吧？要是觉得容易，再来告诉我 GBK 编码的“0xBCC6”表示什么字符。因此计算机与人交互时，一般会把字符转化成人类习惯的表现形式进行输出，如显示、打印的时候。

但是如果仅有字符编码，计算机还不知道该如何表达该字符，因为字符实际上是一个个独特的图形，计算机必须把字符编码转化成对应的字符图形人类才能正常识别，因此我们要给计算机提供字符的图形数据，这些数据就是字模，多个字模数据组成的文件也被称为字库。计算机显示字符时，根据字符编码与字模数据的映射关系找到它相应的字模数据，液晶屏根据字模数据显示该字符。

29.2.1 字模的构成

已知字模是图形数据，而图形在计算机中是由一个个像素点组成的，所以字模实质是一个个像素点数据。为方便处理，我们把字模定义成方块形的像素点阵，且每个像素点只有 0 和 1 这两种状态(可以理解为单色图像数据)。见图 28-35，这是两个宽、高为 16x16 的像素点阵组成的两个汉字图形，其中的黑色像素点即为文字的笔迹。计算机要表示这样的图形，只需使用 16x16 个二进制数据位，每个数据位记录一个像素点的状态，把黑色像素点以“1”表示，无色像素点以“0”表示即可。这样的—个汉字图形，使用 16x16/8=32 个字节来就可以记录下来。

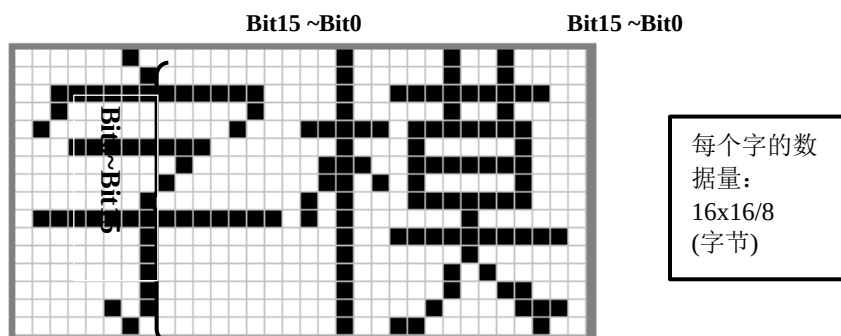


图 28-35 字模

16x16 的“字”的字模数据以 C 语言数组的方式表示，见代码清单 28-18。在这样的字模中，以两个字节表示一行像素点，16 行构成一个字模。

代码清单 28-18: “字”的字模数据

```
1. /* 字 */
2. unsigned char code Bmp003[]=
3. {
4. /*-----
5. ; 源文件 / 文字: 字
6. ; 宽×高(像素): 16×16
7. ; 字模格式/大小: 单色点阵液晶字模, 横向取模, 字节正序/32 字节
8. -----*/
9.
10. 0x02, 0x00, 0x01, 0x00, 0x3F, 0xFC, 0x20, 0x04, 0x40, 0x08, 0x1F, 0xE0, 0x00, 0x40,
    0x00, 0x80,
11. 0xFF, 0xFF, 0x7F, 0xFE, 0x01, 0x00, 0x01, 0x00, 0x01, 0x00, 0x01, 0x00, 0x05, 0x00,
    0x02, 0x00,
12. };
```

29.2.2 字模显示原理

如果使用 LCD 的画点函数，按位来扫描这些字模数据，把为 1 的位以黑色来显示(也可以使用其它颜色)，为 0 的数据位以白色来显示，即可把整个点阵还原出来，显示在液晶屏上。

以便于理解，我们编写了一个使用串口 printf 利用字模打印字符到串口上位机，见代码清单 28-19 中演示的字模显示原理。

代码清单 28-19 使用串口利用字模打印字到上位机

```
1
2
3 /*"当"字符的字模 16x16 */
4 unsigned char charater_matrix[] = {
5     /*"当",0*/
6     0x01,0x00,0x21,0x08,0x11,0x08,0x09,0x10,
7     0x09,0x20,0x01,0x00,0x7F,0xF8,0x00,0x08,
8     0x00,0x08,0x00,0x08,0x3F,0xF8,0x00,0x08,
9     0x00,0x08,0x00,0x08,0x7F,0xF8,0x00,0x08,
10 };
11
12 /**
13  * @brief 使用串口在上位机打印字模
14  *        演示字模显示原理
15  * @retval 无
16  */
17 void Printf_Charater(void)
18 {
19     int i,j;
20     unsigned char kk;
21
22     /*i 用作行计数*/
23     for ( i=0; i<16; i++) {
24         /*j 用作一字节内数据的移位计数*/
25         /*一行像素的第一个字节*/
26         for (j=0; j<8; j++) {
27             /*一个数据位一个数据位地处理*/
28             kk = charater_matrix[2*i] << j ; //左移 j 位
29             if ( kk & 0x80) {
30                 printf("*"); //如果最高位为 1，输出*号，表示笔迹
31             } else {
32                 printf(" "); //如果最高位为 0，输出空格，表示空白
33             }
34         }
35         /*一行像素的第二个字节*/
36         for (j=0; j<8; j++) {
37             kk = charater_matrix[2*i+1] << j ; //左移 j 位
38
39             if ( kk & 0x80) {
40                 printf("*"); //如果最高位为 1，输出*号，表示笔迹
41             } else {
42                 printf(" "); //如果最高位为 0，输出空格，表示空白
43             }
44         }
45         printf("\n"); //输出完一行像素，换行
46     }
47     printf("\n\n"); //一个字输出完毕
48 }
49
```

在 main 函数中运行这段代码，连接好开发板到上位机，可以看到图 28-36 中的现象。该函数中利用 printf 函数对字模数据中为 1 的数据位打印“*”号，为 0 的数据位打印出“空格”，从而在串口接收区域中使用“*”号表达出了一个“当”字。



图 28-36 使用串口打印字模

29.2.3 如何制作字模

以上只是某几个字符的字模，为方便使用，我们需要制作所有常用字符的字模，如程序只需要英文显示，那就需要制作包含 ASCII 码表 28-9 中所有字符的字模，如程序只需要使用一些常用汉字，我们可以选择制作 GB2312 编码里所有字符的字模，而且希望字模数据与字符编码有固定的映射关系，以便在程序中使用字符编码作为索引，查找字模。在网上搜索可找到一些制作字模的软件工具，可满足这些需求。在我们提供的《液晶显示中英文》的工程目录下提供了一个取模软件“PCtoLCD”，这里以它为例讲解如何制作字模，其它字模软件也是类似的。

(1) 配置字模格式

打开取模软件，点击“选项”菜单，会弹出一个对话框，见图 28-37。

- ❑ 选项“点阵格式”中的阴、阳码是指字模点阵中有笔迹像素位的状态是“1”还是“0”，像我们前文介绍的那种就是阴码，反过来就是阳码。本工程中使用阴码。
- ❑ 选项“取模方式”是指字模图形的扫描方向，修改这部分的设置后，选项框的右侧会有相应的说明及动画显示，这里我们依然按前文介绍的字模类型，把它配置成“逐行式”
- ❑ 选项“每行显示的数据”里我们把点阵和索引都配置成 32，设置这个点阵的像素大小为 32x32。

字模选项的格式保持不变，设置完我们点击确定即可，字模选项的这些配置会影响到显示代码的编写方式，即类似前文代码清单 28-19 中的程序。

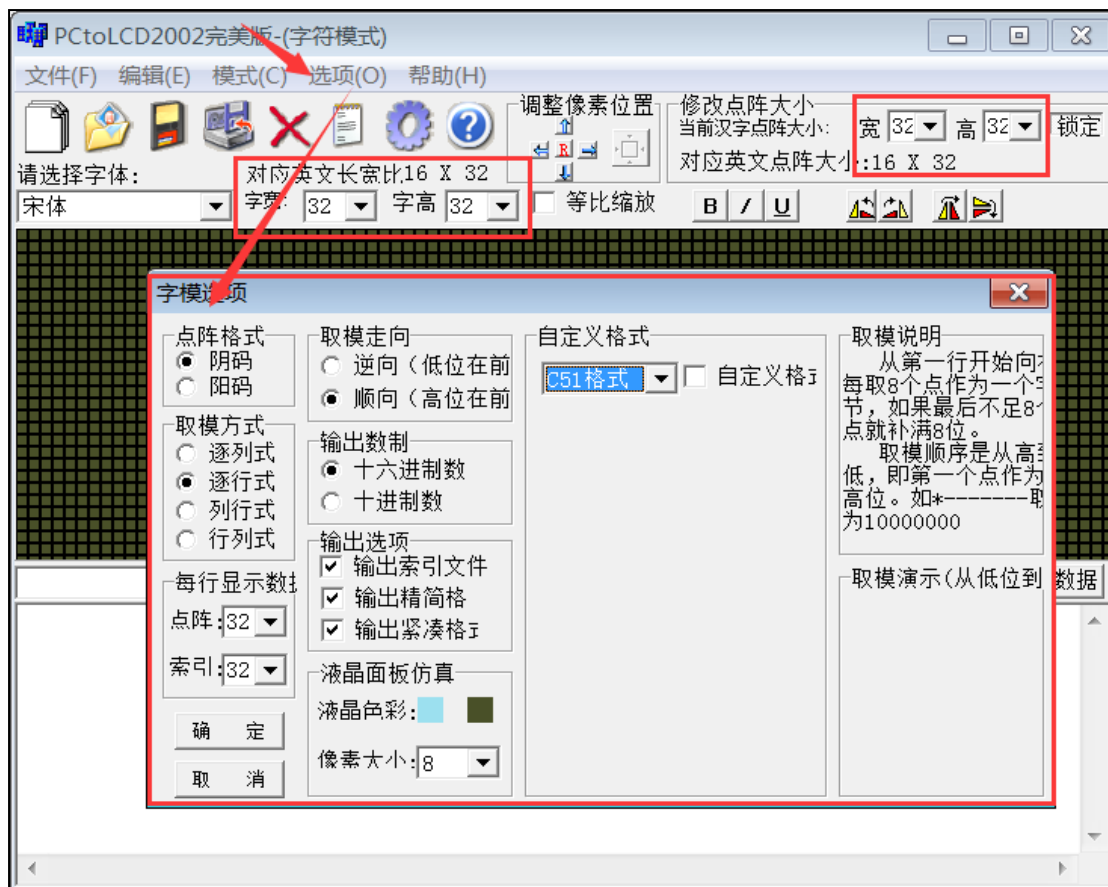


图 28-37 配置字模格式

(2) 生成 GB2312 字模

配置完字模选项后，点击软件中的导入文本图标，会弹出一个“生成字库”的对话框，点击右下角的生成国标汉字库按钮即可生成包含了 GB2312 编码里所有字符的字模文件。在《液晶显示中英文》的工程目录下的《GB2312_H3232.FON》是我用这个取模软件生成的字模原文件，若不想自己制作字模，可直接使用该文件。

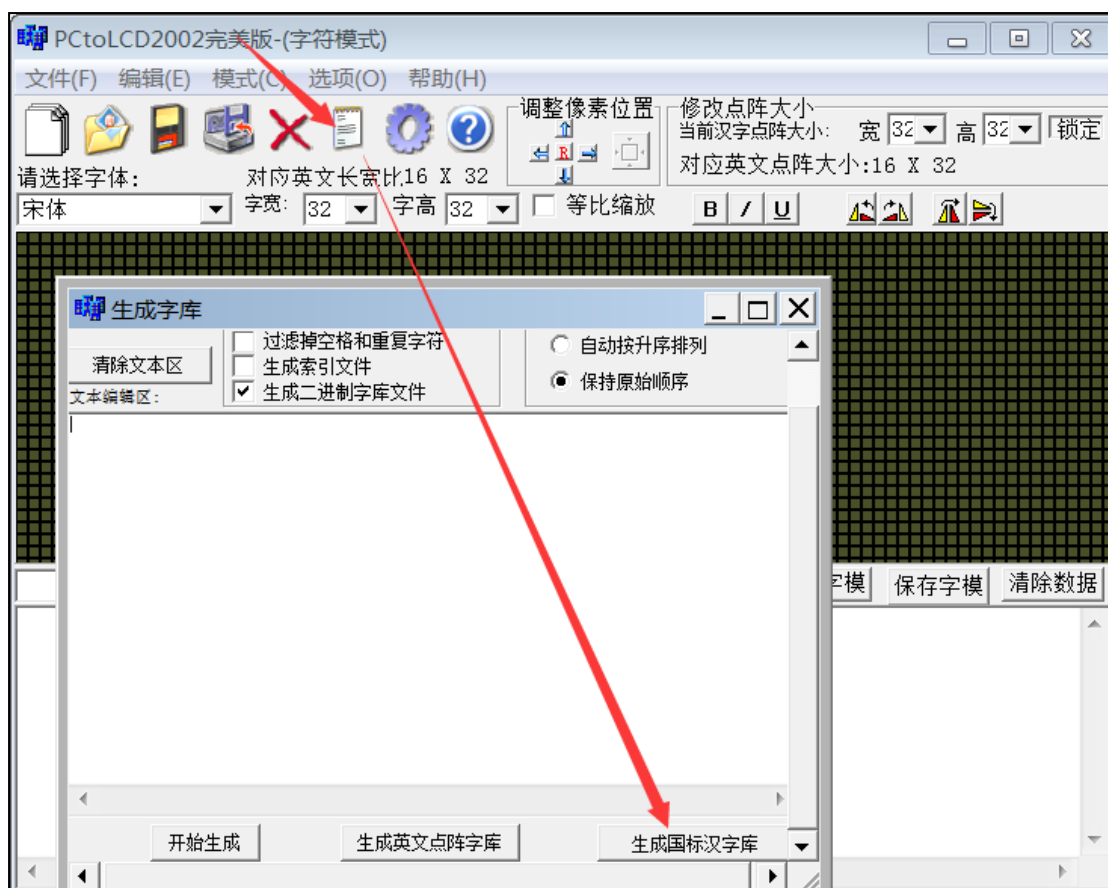


图 28-38 生成国标汉字库

29.2.4 字模寻址公式

使用字模软件制作的字模数据一般会按照编码格式排列。如我们利用以上软件生成的字模文件《GB2312_H3232.FON》中的数据，是根据 GB2312 的区位码表的顺序存储的，它存储了区位码为 0101-9494 的字符，每个字模的大小为 $32 \times 32 / 8 = 128$ 字节。其中第一个字符“空格”的区位码为 0101，它是首个字符，所以文件的前 128 字节存储的是它的字模数据；同理，128-256 字节存储的则是 0102 字符“、”的字模数据。所以我们可以导出任意字符的寻址公式：

$$\text{Addr} = (((\text{Code}_H - 0xA0 - 1) * 94) + (\text{Code}_L - 0xA0 - 1)) * 32 * 32 / 8$$

其中 Code_H 和 Code_L 分别是 GB2312 编码的第一字节和第二字节；94 是指一个区中有 94 个位(即 94 个字符)。公式的实质是根据字符的 GB2312 编码，求出区位码，然后区位码乘以每个字符占据的字节数，求出地址偏移。

29.2.5 存储字模文件

上面生成的《GB2312_H3232.FON》文件的大小为 1023KB，比很多 STM32 芯片内部的所有 FLASH 空间都大，如果我们还是在程序中直接以 C 语言数组的方式存储字模数据，STM32 芯片的程序空间会非常紧张，一般的做法是把字模数据存储到外部存储器，如 SD 卡或 SPI-FLASH 芯片，当需要显示某个字符时，控制器根据字符的编码算好字模的存储地

址，再从存储器中读取，而 FLASH 芯片在生产前就固化好字模内容，然后直接把 FLASH 芯片贴到电路板上，作为整个系统的一部分。

29.3 各种模式的液晶显示字符实验

本小节讲解如何利用字模使用在液晶屏上显示字符。

根据编码或字模存储位置、使用方式的不同，讲解中涉及到多个工程，见表 28-17 中的说明，在讲解特定实验的时候，请读者打开相应的工程来阅读

表 28-17 各种模式的液晶显示字符实验

工程名称	说明
液晶显示	仅包含 ASCII 码字符显示功能，字库直接以 C 语言常量数组的方式存储在 STM32 芯片的内部 FLASH 空间
液晶显示中英文（字库在外部 FLASH）	包含 ASCII 码字符及 GB2312 码字符的显示功能，ASCII 码字符存储在 STM32 内部 FLASH，GB2312 码字符存储在外部 SPI-FLASH 芯片
液晶显示中英文（字库在 SD 卡）	包含 ASCII 码字符及 GB2312 码字符的显示功能，ASCII 码字符存储在 STM32 内部 FLASH，GB2312 码字符直接以文件的格式存储在 SD 卡中
液晶显示中英文（任意大小）	在基础字库的支持下，使用字库缩放函数，使得只用一种字库，就能显示任意大小的字符。包含 ASCII 码字符及 GB2312 码字符的显示功能，ASCII 码字符存储在 STM32 内部 FLASH，GB2312 码字符存储在外部 SPI-FLASH 芯片

这些实验是在《液晶显示》工程的基础上修改的，主要添加字符显示相关的内容，本小节只讲解这部分新增的函数。关于液晶驱动的原理在此不再赘述，不理解这部分的可阅读前面的章节。

29.3.1 硬件设计

针对不同模式的液晶显示字符工程，需要有不同的硬件支持。字库存储在 STM32 芯片内部 FLASH 的工程跟普通液晶显示的硬件需求无异。需要外部字库的工程，要有额外的 SPI-FLASH、SD 支持，使用外部 FLASH 时，我们的实验板上直接用板子上的 SPI-FLASH 芯片存储字库，出厂前我们已给 FLASH 芯片烧录了前面的《GB2312_H3232.FON》字库文件，如果您想把我们的程序移植到您自己设计产品上，请确保该系统包含有存储了字库的 FLASH 芯片，才能正常显示汉字。使用 SD 卡时，需要给板子接入存储有《GB2312_H3232.FON》字库文件的 MicroSD 卡，SD 卡的文件系统格式需要是 FAT 格式，且字库文件所在的目录需要跟程序里使用的文件目录一致。

关于 SPI-FLASH 和 SD 卡的原理图及驱动说明可参考其他的章节。给外部 SPI-FLASH 和 SD 卡存储字库的操作我们将在另一个文档中说明，本章的教程默认您已配置好 SPI-FLASH 和 SD 卡相关的字库环境。

29.3.2 显示 ASCII 编码的字符

我们先来看如何显示 ASCII 码表中的字符，请打开“**液晶显示**”（即上一章节的例程）的工程文件。本工程中我们把字库数据相关的函数代码写在“**fonts.c**”及“**fonts.h**”文件中，字符显示的函数仍存储在 LCD 驱动文件“**bsp_ili9806g_lcd.c**”及“**bsp_ili9806g_lcd.h**”中。

1. 编程要点

- (1) 获取字模数据；
- (2) 根据字模格式，编写液晶显示函数；
- (3) 编写测试程序，控制液晶英文。

2. 代码分析

ASCII 字模数据

要显示字符首先要有字库数据，在工程的“**fonts.c**”文件中我们定义了一系列大小为 24x48、16x32、8x16 的 ASCII 码表的字模数据，其形式见代码清单 28-20。

代码清单 28-20 部分英文字库 16x32 大小(fonts.c 文件)

```
1
2 /*
3  * 常用 ASCII 表，偏移量 32，大小:32（高度）* 16（宽度）
4  */
5 // @conslons 字体，阴码点阵格式，逐行顺向取模
6 const uint8_t ASCII16x32_Table [ ] = {
7     0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
8     0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
9     0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
10    0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
11    0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
12    0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
13    0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
14    0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
15    0x00,0x00, 0x00,0x00, 0x00,0x00, 0x00,0x00,
16    0x00,0x00, 0x00,0x00, 0x03,0x80, 0x03,0x80,
17    0x01,0x80, 0x01,0x80, 0x01,0x80, 0x01,0x80,
18    0x01,0x80, 0x01,0x80, 0x01,0x80, 0x01,0x80,
19    0x01,0x80, 0x01,0x80, 0x01,0x80, 0x01,0x80,
20    0x00,0x00, 0x00,0x00, 0x03,0x80, 0x03,0xc0,
21
22    /*以下部分省略.....，包含从 空格 至 波浪号 的 ASCII 码图形字模数据*/
23
```

由于 ASCII 中的字符并不多，所以本工程中直接以 C 语言数组的方式存储这些字模数据，C 语言的 const 数组是作为常量直接存储到 STM32 芯片的内部 FLASH 中的，所以如果您不需要显示中文，可以不用外部的 SPI-FLASH 芯片，可省去烧录字库的麻烦。以上代码定义的 ASCII16x32_Table 数组是 16x32 大小的 ASCII 字库。

管理英文字模的结构体

为了方便使用各种不同的字体，工程中定义了一个“sFont”结构体类型，并利用它定义存储了不同字体信息的结构体变量，见代码清单 28-21。

代码清单 28-21 管理英文字模的结构体(fonts.c 文件)

```
1
2 /*字体格式*/
3 typedef struct _tFont {
4     const uint8_t *table; /*指向字模数据的指针*/
5     uint16_t Width; /*字模的像素宽度*/
6     uint16_t Height; /*字模的像素高度*/
7 } sFONT;
8
9
10 sFONT Font8x16 = {
11     ASCII8x16_Table,
12     8, /* 字模宽 */
13     16, /* 字模高 */
14 };
15
16 sFONT Font16x32 = {
17     ASCII16x32_Table,
18     16, /* 字模宽 */
19     32, /* 字模高 */
20 };
21
22 sFONT Font24x48 = {
23     ASCII24x48_Table,
24     24, /* 字模宽 */
25     48, /* 字模高 */
26 };
```

这个结构体类型定义了一个变量，第一个是指向字模数据的指针，即前面提到的 C 语言数组，每二、三个变量存储了该字模单个字符的像素宽度和高度。利用这个类型定义了 Font8x16、Font16x32 之类的变量，方便显示时寻址。

切换字体

在程序中若要方便切换字体，还需要定义一个存储了当前选择字体的变量 LCD_Currentfonts，代码清单 28-22。

代码清单 28-22 切换字体(bsp_lcd.c 文件)

```
1 /*用于存储当前选择的字体格式的全局变量*/
2 static sFONT *LCD_Currentfonts;
3 /**
4  * @brief 设置字体格式(英文)
5  * @param fonts: 选择要设置的字体格式
6  * @retval None
7  */
8 void LCD_SetFont(sFONT *fonts)
9 {
10     LCD_Currentfonts = fonts;
11 }
```

使用 LCD_SetFont 可以切换 LCD_Currentfonts 指向的字体类型，函数的可输入参数即前面的 Font8x16、Font16x32 之类的变量。

ASCII 字符显示函数

利用字模数据以及上述结构体变量，我们可以编写一个能显示各种字体的通用函数，见代码清单 28-23。

代码清单 28-23 ASCII 字符显示函数

```
1
2 /**
3  * @brief 在 ILI9806G 显示器上显示一个英文字符
```

```
4  * @param usX : 在特定扫描方向下字符的起始 X 坐标
5  * @param usY : 在特定扫描方向下该点的起始 Y 坐标
6  * @param cChar : 要显示的英文字符
7  * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
8  * @retval 无
9  */
10 void ILI9806G_DispcChar_EN ( uint16_t usX, uint16_t usY, const char cChar )
11 {
12     uint8_t byteCount, bitCount, fontLength;
13     uint16_t ucRelativePositon;
14     uint8_t *Pfont;
15
16     //对 ascii 码表偏移 (字模表不包含 ASCII 表的前 32 个非图形符号)
17     ucRelativePositon = cChar - ' ';
18
19     //每个字模的字节数
20     fontLength = (LCD_Currentfonts->Width*LCD_Currentfonts->Height)/8;
21
22     //字模首地址
23     /*ascii 码表偏移值乘以每个字模的字节数, 求出字模的偏移位置*/
24     Pfont = (uint8_t *)&LCD_Currentfonts->table[ucRelativePositon * fontLength];
25
26     //设置显示窗口
27     ILI9806G_OpenWindow ( usX, usY, LCD_Currentfonts->Width, LCD_Currentfonts->Height);
28
29     ILI9806G_Write_Cmd ( CMD_SetPixel );
30
31     //按字节读取字模数据
32     //由于前面直接设置了显示窗口, 显示数据会自动换行
33     for ( byteCount = 0; byteCount < fontLength; byteCount++ ) {
34         //一位一位处理要显示的颜色
35         for ( bitCount = 0; bitCount < 8; bitCount++ ) {
36             if ( Pfont[byteCount] & (0x80>>bitCount) )
37                 ILI9806G_Write_Data ( CurrentTextColor );
38             else
39                 ILI9806G_Write_Data ( CurrentBackColor );
40         }
41     }
42 }
```

这个函数与前文中的串口打印字模到上位机的那个函数原理是一样的, 只是这个函数要使用液晶显示, 且字模数据并不是一个个独立的数组, 而是所有字符的字模都放到同一个数组里, 所以显示时, 要根据字符编码计算字模数据的偏移, 并把串口打印的处理改成像素点显示。该函数的说明如下:

(1) 输入参数

这个字符显示函数有 usX、usY 及 cChar 参数。其中 usX 和 usY 分别表示字符显示位置的(X,Y)坐标; 而输入参数 cChar 是要显示的英文字符, 如字符'A', 字符'空格'等。

(2) 根据字符计算字模的数组偏移

在显示前, 首先要提取出字符相应的字模数据。由于 ASCII 码中的编码 0~31(NULL~US 符号)是不存在图形表示的, 为节省空间, 字模表中只包含图形符号相关的数据, 例如, 对于 Font8x16 的字模表, 每个字符的字模数据长度=8*16/8=16 字节, 那么 0~15 字节表示的是'空格'字模, 16~31 字节表示的是'!'号字模, 32~47 字节表示的是'“'号字模。

因此, 显示函数中通过 cChar 获知要显示的字符后, 使用 ucRelativePositon 变量存储 cChar 减去字符'空格'的 ASCII 码值, 即求出图形 ASCII 码的偏移值; 然后使用

fontLength 变量存储根据当前选择的字体宽度与高度计算出的单个字模数据长度；最后根据它在字模数组表中的偏移值（ucRelativePositon 与 fontLength 的乘积）求出该字模的地址指针，存储在指针变量 Pfont 中，后面就可以直接使用该指针获取字模数据了，从 Pfont 至(Pfont+fontLength-1)范围内的都是输入字符 cChar 的字模数据。

(3) 设置显示窗口并发送显示命令

计算出字模数据的指针后，可以准备开始显示，函数中使用 ILI9806G_OpenWindow 函数根据输入的显示坐标及字模的宽高设置一个字符的显示窗口，并使用 ILI9806G_Write_Cmd 函数发送设置像素点的命令（CMD_SetPixel），有了这两个操作后，下面使用的 ILI9806G_Write_Data 函数发送的像素点数据将会一行一行地显示到窗口中（沿 X 方向，到达 X 结尾后沿 Y 方向显示下一行），见图 28-39。

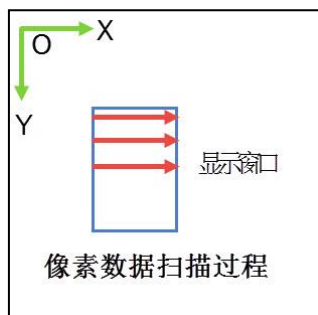


图 28-39 设置显示窗口后的像素数据扫描过程

(4) 行循环与列循环

由于根据字模大小设置了显示窗口，使用 ILI9806G_Write_Data 函数发送像素数据时到达单行的结尾它会自动换行，所以在发送数据时不需要再考虑换行。代码中直接使用两层循环处理字模数据，其中外层 for 循环用于遍历字模的字节数据，一个字节一个字节地读取，而字节数据的处理则交给内层 for 循环，当外层 for 循环遍历完 fontLength 个字节表示处理完一个字符的字模，即显示完一个字符；内层 for 循环用于遍历字模单个字节数据的每个数据位，数据位为 1 时就发送一个点的字体颜色（CurrentTextColor），数据位为 0 时就发送背景颜色（CurrentBackColor）。

经过 ILI9806G_DispChar_EN 函数的处理，可显示一个英文字符，要显示字符串时，重复调用本函数即可。

显示字符串

对 ILI9806G_DispChar_EN 函数进行封装，我们可以得到 ASCII 字符的字符串显示函数，见代码清单 28-24。

代码清单 28-24 字符串显示函数

```
1 /**
2  * @brief 在 ILI9806G 显示器上显示英文字符串
3  * @param usX : 在特定扫描方向下字符的起始 X 坐标
4  * @param usY : 在特定扫描方向下字符的起始 Y 坐标
5  * @param pStr : 要显示的英文字符串的首地址
6  * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
7  * @retval 无
```



```
8  */
9 void ILI9806G_DispString_EN ( uint16_t usX ,uint16_t usY,  char * pStr )
10 {
11     while ( * pStr != '\0' ) {
12 if ( ( usX - ILI9806G_DispWindow_X_Star + LCD_Currentfonts->Width )> LCD_X_LENGTH) {
13         usX = ILI9806G_DispWindow_X_Star;
14         usY += LCD_Currentfonts->Height;
15     }
16
17 if ( ( usY -ILI9806G_DispWindow_Y_Star + LCD_Currentfonts->Height )>LCD_Y_LENGTH ) {
18     usX = ILI9806G_DispWindow_X_Star;
19     usY = ILI9806G_DispWindow_Y_Star;
20 }
21
22     ILI9806G_DispChar_EN ( usX, usY, * pStr);
23
24     pStr ++;
25     usX += LCD_Currentfonts->Width;
26 }
27 }
```

本函数中的输入参数 `pStr` 为指向要显示的字符串的指针，在函数的内部利用 `while` 循环把字符串中的字符一个个地利用 `ILI9806G_DispChar_EN` 函数显示到液晶屏上，当遇到字符串结束符`\0`时完成显示，退出 `while` 循环，结束函数。在 `while` 循环的开头，有两个 `if` 判断操作，它们分别用于判断显示字符时的 `X` 及 `Y` 坐标是否超出屏幕的边沿，若超出了则换到下一行，使用这个函数，可以很方便地利用 “`ILI9806G_DispString_EN (10, 20,“test”)`” 这样的格式在液晶屏上直接显示一串字符。

使用宏计算 Y 坐标

使用 `ILI9806G_DispString_EN` 函数显示时，需要注意字 `Y` 方向字符覆盖的问题，例如在(10,20)坐标处显示了一行字体高度为 16 像素的字符串，如果再显示另一串字符时指定的坐标为(10,25)，那么由于高度预留不足，会出现字符覆盖的现象，因此调用时需要小心计算 `Y` 方向的坐标。为了简化操作，代码中定义了一个宏 `LINE` 及函数 `ILI9806G_DispStringLine_EN` 来显示字符串，见代码清单 28-25。

代码清单 28-25 使用宏计算 Y 坐标

```
1
2 #define LINE(x) ((x) * (((sFONT *)LCD_GetFont())->Height))
3
4 /**
5  * @brief 在 ILI9806G 显示器上显示英文字符串
6  * @param line : 在特定扫描方向下字符串的起始 Y 坐标
7  * 本参数可使用宏 LINE(0)、LINE(1) 等方式指定文字坐标，
8  * 宏 LINE(x) 会根据当前选择的字体来计算 Y 坐标值。
9  * 显示中文且使用 LINE 宏时，需要把英文字体设置成 Font8x16
10 * @param pStr : 要显示的英文字符串的首地址
11 * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
12 * @retval 无
13 */
14 void ILI9806G_DispStringLine_EN ( uint16_t line,  char * pStr )
15 {
16     uint16_t usX = 0;
17
18     while ( * pStr != '\0' ) {
19 if(( usX - ILI9806G_DispWindow_X_Star + LCD_Currentfonts->Width ) > LCD_X_LENGTH ) {
20         usX = ILI9806G_DispWindow_X_Star;
21         line += LCD_Currentfonts->Height;
22     }
23 }
```

```
23
24 if((line - ILI9806G_DispWindow_Y_Star + LCD_Currentfonts->Height )> LCD_Y_LENGTH ) {
25     usX = ILI9806G_DispWindow_X_Star;
26     line = ILI9806G_DispWindow_Y_Star;
27 }
28
29 ILI9806G_DispChar_EN ( usX, line, * pStr);
30
31 pStr ++;
32
33 usX += LCD_Currentfonts->Width;
34 }
35 }
```

本函数主体与 ILI9806G_DispString_EN 差异不大，主要是删减了 X 方向的坐标，另外使用输入参数 line 来指定 Y 方向的坐标，调用本函数时，一般配合上面的 LINE 宏来使用，该宏会根据当前选择的字体高度来计算字符 Y 方向的间隔，如当前字体为 8x16 时，字体高度为 16 像素，所以 LINE(1)会返回数值 16，LINE(3)会返回数值 48。利用它配合本函数可以使用“ILI9806G_DispStringLine_EN (LINE(1),”test”);”的形式来使字符串显示在第 1 行，即 Y 方向为 16 像素处，当然，使用前面的函数“ILI9806G_DispString_EN (10,LINE(1),”test”);”同样也是可以的，并且可指定 X 坐标。

清除屏幕字符

在实际应用中，还经常需要把当前屏幕显示的内容清除掉，这可以使用代码清单 28-26 中的函数。

代码清单 28-26 清除屏幕字符

```
1 /**
2  * @brief 对 ILI9806G 显示器的某一窗口以某种颜色进行清屏
3  * @param usX : 在特定扫描方向下窗口的起点 X 坐标
4  * @param usY : 在特定扫描方向下窗口的起点 Y 坐标
5  * @param usWidth : 窗口的宽度
6  * @param usHeight : 窗口的高度
7  * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
8  * @retval 无
9  */
10 void ILI9806G_Clear ( uint16_t usX, uint16_t usY,
11                       uint16_t usWidth, uint16_t usHeight )
12 {
13     ILI9806G_OpenWindow ( usX, usY, usWidth, usHeight );
14     ILI9806G_FillColor ( usWidth * usHeight, CurrentBackColor );
15 }
16
17
18
19 /**
20  * @brief 清除某行文字
21  * @param Line: 指定要删除的行
22  * 本参数可使用宏 LINE(0)、LINE(1)等方式指定要删除的行，
23  * 宏 LINE(x)会根据当前选择的字体来计算 Y 坐标值，并删除当前字体高度的第 x 行。
24  * @retval None
25  */
26 void LCD_ClearLine(uint16_t Line)
27 { /* 清屏，显示背景颜色 */
28     ILI9806G_Clear(0,Line,LCD_X_LENGTH,((sFONT *)LCD_GetFont())->Height);
29 }
30 }
```

代码中的 ILI9806G_Clear 函数可以直接清除一个指定的矩形，它会把该矩形显示成当前设置的背景颜色 CurrentBackColor，实现清除图像的效果，而 LCD_ClearLine 函数对它进行了封装，使用 LINE 宏配合，可以比较方便地清除单行字符串，如调用 LCD_ClearLine(LINE(1))可以清除第 1 行的字符串。

显示 ASCII 码示例

下面我们再来看 main 文件是如何利用这些函数显示 ASCII 码字符的，见代码清单 28-27。

代码清单 28-27 显示 ASCII 码的 main 函数

```
1  /**
2   * @brief 主函数
3   * @param 无
4   * @retval 无
5   */
6  int main ( void )
7  {
8      ILI9806G_Init ();          //LCD 初始化
9
10     USART_Config();
11
12     printf("\r\n ***** 液晶屏英文显示程序***** \r\n");
13     printf("\r\n 本程序不支持中文，显示中文的程序请学习下一章 \r\n");
14
15     //其中 0、3、5、6 模式适合从左至右显示文字，
16     //不推荐使用其它模式显示文字 其它模式显示文字会有镜像效果
17     //其中 6 模式为大部分液晶例程的默认显示方向
18     ILI9806G_GramScan ( 6 );
19     while ( 1 ) {
20         LCD_Test();
21     }
22 }
```

main 函数中主要是对液晶屏初始化，初始化完成后就能够显示 ASCII 码字符了，无需利用 SPI-FLASH 及 SD 卡。在 while 循环中调用的 LCD_Test 函数包含了显示字符的函数调用示例，见代码清单 28-28。

代码清单 28-28 LCD_Test 函数中的 ASCII 码显示示例

```
1
2
3  /*用于测试各种液晶的函数*/
4  void LCD_Test(void)
5  {
6      /*演示显示变量*/
7      static uint8_t testCNT = 0;
8      char dispBuff[100];
9
10     testCNT++;
11
12     LCD_SetFont(&Font16x32);
13     LCD_SetColors(RED,BLACK);
14
15     ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏，显示全黑 */
16     /******显示字符串示例******/
17     ILI9806G_DispStringLine_EN(LINE(0),"BH 4.5 inch LCD");
18     ILI9806G_DispStringLine_EN(LINE(2),"resolution:480x854px");
19     ILI9806G_DispStringLine_EN(LINE(3),"LCD driver:ILI9806G");
20     ILI9806G_DispStringLine_EN(LINE(4),"Touch driver:GT5688");
```

```
21
22     /*****显示变量示例*****/
23     LCD_SetTextColor(GREEN);
24
25     /*使用 c 标准库把变量转化成字符串*/
26     sprintf(disBuff, "Count : %d ", testCNT);
27     ILI9806G_ClearLine(LINE(7)); /* 清除单行文字 */
28
29     /*然后显示该字符串即可，其它变量也是这样处理*/
30     ILI9806G_DispStringLine_EN(LINE(7), disBuff);
31     /*以下省略其它液晶演示示例*/
32 }
```

这段代码包含了使用字符串显示函数显示常量字符和变量的示例。显示常量字符串时，直接使用双引号括起要显示的字符串即可，根据 C 语言的语法，这些字符串会被转化成常量数组，数组内存储对应字符的 ASCII 码，然后存储到 STM32 的 FLASH 空间，函数调用时通过指针来找到对应的 ASCII 码，液晶显示函数使用前面分析过的流程，转换成液晶显示输出。

在很多场合下，我们可能需要使用液晶屏显示代码中变量的内容，这时很多用户就不知道该如何解决了，上面的 LCD_Test 函数结尾处演示了如何处理。它主要是使用一个 C 语言标准库里的函数 sprintf，把变量转化成 ASCII 码字符串，转化后的字符串存储到一个数组中，然后我们再利用液晶显示字符串的函数显示该数组的内容即可。sprintf 函数的用法与 printf 函数类似，使用它时需要包含头文件 string.h。

29.3.3 显示 GB2312 编码的字符

显示 ASCII 编码比较简单，由于字库文件小，甚至都不需要使用外部的存储器，而显示汉字时，由于我们的字库是存储到外部存储器上的，这涉及到额外的获取字模数据的操作。

我们分别制作了两个工程来演示如何显示汉字，以下部分的内容请打开“**液晶显示中英文（字库在外部 FLASH）**”和“**液晶显示中英文（字库在 SD 卡）**”工程阅读理解。这两个工程使用的字库文件内容相同，只是字库存储的位置不一样，工程中我们把获取字库数据相关的函数代码写在“fonts.c”及“fonts.h”文件中，字符显示的函数仍存储在 LCD 驱动文件“bsp_ili9806g_lcd.c”及“bsp_ili9806g_lcd.h”中。

1. 编程要点

- (1) 获取字模数据；
- (2) 根据字模格式，编写液晶显示函数；
- (3) 编写测试程序，控制液晶汉字。

2. 代码分析

显示汉字字符

由于我们的 GB2312 字库文件与 ASCII 字库文件是使用同一种方式生成的，但字符的编码不同导致字模偏移地址计算有区别，且字模数据存储的位置不同，所以为了显示汉字，

需要另外编写一个字符显示函数，它利用前文生成的《GB2312_H3232.FON》字库显示 GB2312 编码里的字符，见代码清单 28-9。

代码清单 28-29 显示 GB2312 编码字符的函数(bsp_ili9806g_ldc.c 文件)

```
1
2 #define          WIDTH_CH_CHAR          32          //中文字符宽度
3 #define          HEIGHT_CH_CHAR         32          //中文字符高度
4
5 //缓存读取回来的字模数据
6 static uint8_t ucBuffer [ WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8 ];
7
8
9 /**
10 * @brief 在 ILI9806G 显示器上显示一个中文字符
11 * @param usX : 在特定扫描方向下字符的起始 X 坐标
12 * @param usY : 在特定扫描方向下字符的起始 Y 坐标
13 * @param usChar : 要显示的中文字符（国标码）
14 * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
15 * @retval 无
16 */
17 void ILI9806G_DispChar_CH ( uint16_t usX, uint16_t usY, uint16_t usChar )
18 {
19     uint8_t rowCount, bitCount;
20     uint32_t usTemp;
21
22     // 占用空间太大，改成全局变量
23     // uint8_t ucBuffer [ WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8 ];
24
25     //设置显示窗口
26     ILI9806G_OpenWindow ( usX, usY, WIDTH_CH_CHAR, HEIGHT_CH_CHAR );
27
28     ILI9806G_Write_Cmd ( CMD_SetPixel );
29
30     //取字模数据
31     GetGBKCode ( ucBuffer, usChar );
32
33     for ( rowCount = 0; rowCount < HEIGHT_CH_CHAR; rowCount++ ) {
34         /* 取出四个字节的数据，在 lcd 上即是一个汉字的一行 */
35         usTemp = ucBuffer [ rowCount * 4 ];
36         usTemp = ( usTemp << 8 );
37         usTemp |= ucBuffer [ rowCount * 4 + 1 ];
38         usTemp = ( usTemp << 8 );
39         usTemp |= ucBuffer [ rowCount * 4 + 2 ];
40         usTemp = ( usTemp << 8 );
41         usTemp |= ucBuffer [ rowCount * 4 + 3 ];
42
43         for ( bitCount = 0; bitCount < WIDTH_CH_CHAR; bitCount ++ ) {
44             if ( usTemp & ( 0x80000000 >> bitCount ) ) //高位在前
45                 ILI9806G_Write_Data ( CurrentTextColor );
46             else
47                 ILI9806G_Write_Data ( CurrentBackColor );
48         }
49     }
50 }
```

这个 GB2312 码的显示函数与 ASCII 码的显示函数是很类似的，它的输入参数有 usX, usY 及 usChar。其中 usX 和 usY 用于设定字符的显示位置，usChar 是字符的编码，这是一个 16 位的变量，因为 GB2312 编码中每个字符是 2 个字节的。函数的执行流程介绍如下：

- (1) 使用 ILI9806G_OpenWindow 和 ILI9806G_Write_Cmd 来设置显示窗口并发送显示像素命令。

- (2) 使用量 `macGetGBKCode` 函数获取字模数据, 向该函数输入 `usChar` 参数(字符的编码), 它会从外部 SPI-FLASH 芯片或 SD 卡中读取该字符的字模数据, 读取得的数据被存储到数组 `ucBuffer` 中。关于 `GetGBKCode` 函数我们在后面详细讲解。
- (3) 遍历像素点。这个代码在遍历时还使用了 `usTemp` 变量用来缓存一行的字模数据(本字模一行有 4 个字节), 然后一位一位地判断这些数据, 数据位为 1 的时候, 像素点就显示字体颜色, 否则显示背景颜色。原理是跟 ASCII 字符显示一样的。

显示中英文字符串

类似地, 我们希望汉字也能直接以字符串的形式来调用函数显示, 而且最好是中英文字符可以混在一个字符串里。为此, 我们编写了 `LCD_DisplayStringLine_EN_CH` 函数, 代码清单 28-10。

代码清单 28-30 显示中英文的字符串

```
1
2 /**
3  * @brief 在 ILI9806G 显示器上显示中英文字符串
4  * @param usX : 在特定扫描方向下字符的起始 X 坐标
5  * @param usY : 在特定扫描方向下字符的起始 Y 坐标
6  * @param pStr : 要显示的字符串的首地址
7  * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
8  * @retval 无
9  */
10 void ILI9806G_DisplayString_EN_CH (uint16_t usX , uint16_t usY, char * pStr )
11 {
12     uint16_t usCh;
13     while ( * pStr != '\0' ) {
14         if ( * pStr <= 126 ) { //英文字符
15             if( ( usX - ILI9806G_DispWindow_X_Star + LCD_Currentfonts->Width ) > LCD_X_LENGTH ) {
16                 usX = ILI9806G_DispWindow_X_Star;
17                 usY += LCD_Currentfonts->Height;
18             }
19             ILI9806G_DisplayChar_EN ( usX, usY, * pStr );
20             usX += LCD_Currentfonts->Width;
21             pStr++;
22         }
23         else { //汉字字符
24             if ( ( usX - ILI9806G_DispWindow_X_Star + WIDTH_CH_CHAR ) > LCD_X_LENGTH ) {
25                 usX = ILI9806G_DispWindow_X_Star;
26                 usY += HEIGHT_CH_CHAR;
27             }
28             if ( ( usY - ILI9806G_DispWindow_Y_Star + HEIGHT_CH_CHAR ) > LCD_Y_LENGTH ) {
29                 usX = ILI9806G_DispWindow_X_Star;
30                 usY = ILI9806G_DispWindow_Y_Star;
31             }
32             usCh = * ( uint16_t * ) pStr;
33             usCh = ( usCh << 8 ) + ( usCh >> 8 );
34         }
35     }
36 }
```



```
47         ILI9806G_DisPChar_CH ( usX, usY, usCh );
48
49         usX += WIDTH_CH_CHAR;
50         pStr += 2;           //一个汉字两个字节
51     }
52 }
53 }
```

这个函数根据字符串中的编码值，判断它是 ASCII 码还是国标码中的字符，然后作不同处理。英文部分与前方中的英文字符串显示函数是一样的，中文部分也很类似，需要注意的是中文字符每个占 2 个字节，而且由于 STM32 芯片的数据是小端格式存储的，国标码是大端格式存储的，所以函数中对输入参数 pStr 指针获取的编码 usCh 交换了字节顺序，再输入到单个字符的显示函数 LCD_DisPChar_CH 中。

获取 SPI-FLASH 中的字模数据

前面提到的 GetGBKCode 函数用于获取汉字字模数据，它根据字库文件的存储位置，有 SPI-FLASH 和 SD 卡两个版本，我们先来分析比较简单的 SPI-FLASH 版本，代码清单 28-31。该函数定义在“液晶显示中英文（字库在外部 FLASH）”工程的“fonts.c”和“fonts.h”文件中。

代码清单 28-31 从 SPI-FLASH 获取字模数据(“液晶显示中英文（字库在外部 FLASH）”工程)

```
1  /*****fonts.h 文件中的定义 *****/
2  /*使用 FLASH 字模*/
3  /*中文字库存储在 FLASH 的起始地址*/
4  /*FLASH*/
5  #define GBKCODE_START_ADDRESS    1254*4096
6
7
8  /*获取字库的函数*/
9  //定义获取中文字符字模数组的函数名，ucBuffer 为存放字模数组名，usChar 为中文字符（国标码）
10
11 #define      GetGBKCode( ucBuffer, usChar )  GetGBKCode_from_EXFlash( ucBuffer, usChar )
12 int GetGBKCode_from_EXFlash( uint8_t * pBuffer, uint16_t c );
13
14 /*****fonts.c 文件中的字义 *****/
15
16 /*使用 FLASH 字模*/
17 //字模 GB2312_H3232 配套的函数
18
19 //中文字库存储在 FLASH 的起始地址：
20 //GBKCODE_START_ADDRESS 在 fonts.h 文件定义
21 /**
22  * @brief  获取 FLASH 中文显示字库数据
23  * @param  pBuffer:存储字库矩阵的缓冲区
24  * @param  c : 要获取的文字
25  * @retval None.
26  */
27 int GetGBKCode_from_EXFlash( uint8_t * pBuffer, uint16_t c )
28 {
29     unsigned char High8bit,Low8bit;
30     unsigned int pos;
31
32     static uint8_t everRead=0;
33
34     /*第一次使用，初始化 FLASH*/
35     if (everRead == 0) {
36         SPI_FLASH_Init();
37         everRead = 1;
38     }
```

```
39
40     High8bit= c >> 8;      /* 取高 8 位数据 */
41     Low8bit= c & 0x00FF;   /* 取低 8 位数据 */
42
43     /*GB2312 公式*/
44     pos = ((High8bit-0xa1)*94+Low8bit-0xa1)*WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8;
45     SPI_FLASH_BufferRead(pBuffer,GBKCODE_START_ADDRESS+pos,
46                          WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8); //读取字库数据
47
48     return 0;
49 }
```

这个 GetGBKCode 实质上是一个宏，当使用 SPI-FLASH 作为字库数据源时，它等效于 GetGBKCode_from_EXFlash 函数，它的执行过程如下：

- (1) 初始化 SPI 外设，以确保后面使用 SPI 读取 FLASH 内容时 SPI 已正常工作，并且初始化后做一个标记，以后再读取字模数据的时候就不需要再次初始化 SPI 了；
- (2) 取出要显示字符的 GB2312 编码的高位字节和低位字节，以便后面用于计算字符的字模地址偏移；
- (3) 根据字符的编码及字模的大小导出的寻址公式，计算当前要显示字模数据在字库中的地址偏移；
- (4) 利用 SPI_FLASH_BufferRead 函数，从 SPI-FLASH 中读取该字模的数据，输入参数中的 GBKCODE_START_ADDRESS 是在代码头部定义的一个宏，它是字库文件存储在 SPI-FLASH 芯片的基地址，该基地址加上字模在字库中的地址偏移，即可求出字模在 SPI-FLASH 中存储的实际位置。这个基地址具体数值是在我们烧录 FLASH 字库时决定的，程序中定义的是实验板出厂时默认烧录的位置。
- (5) 获取到的字模数据存储在 pBuffer 指针指向的存储空间，显示汉字的函数直接利用它来显示字符。

获取 SD 卡中的字模数据

类似地，从 SD 卡中获取字模数据时，使用 GetGBKCode_from_sd 函数，见代码清单 28-32。该函数定义在“液晶显示中英文（字库在 SD 卡）”工程的“fonts.c”和“fonts.h”文件中。

代码清单 28-32 从 SD 卡中获取字模数据（“液晶显示中英文（字库在 SD 卡）”工程）

```
1  /*****fonts.h 文件中的定义 *****/
2  /*使用 SD 字模*/
3
4
5  /*SD 卡字模路径*/
6  #define GBKCODE_FILE_NAME      "0:/Font/GB2312_H3232.FON"
7
8
9  /*获取字库的函数*/
10 //定义获取中文字符字模数组的函数名，ucBuffer 为存放字模数组名，usChar 为中文字符（国标码）
11
12
13 #define GetGBKCode( ucBuffer, usChar )  GetGBKCode_from_sd( ucBuffer, usChar )
14 int GetGBKCode_from_sd ( uint8_t * pBuffer, uint16_t c );
15
16 /*****fonts.c 文件中的字义 *****/
17 /*使用 SD 字模*/
18
```

```
19 static FIL fnew; /* file objects */
20 static FATFS fs; /* Work area (file system object) for logical drives */
21 static FRESULT res_sd;
22 static UINT br; /* File R/W count */
23
24 //字库文件存储位置, fonts.h 中的宏:
25 // #define GBKCODE_FILE_NAME "0:/Font/GB2312_H3232.FON"
26
27 /**
28  * @brief 获取 SD 卡中文显示字库数据
29  * @param pBuffer: 存储字库矩阵的缓冲区
30  * @param c: 要获取的文字
31  * @retval None.
32  */
33 int GetGBKCode_from_sd ( uint8_t * pBuffer, uint16_t c )
34 {
35     unsigned char High8bit, Low8bit;
36     unsigned int pos;
37
38     static uint8_t everRead = 0;
39
40     High8bit = c >> 8; /* 取高 8 位数据 */
41     Low8bit = c & 0x00FF; /* 取低 8 位数据 */
42
43     pos = ((High8bit-0x01)*94+Low8bit-0x01)*WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8;
44
45     /*第一次使用, 挂载文件系统, 初始化 sd*/
46     if (everRead == 0) {
47         res_sd = f_mount(&fs, "0:", 1);
48         everRead = 1;
49     }
50
51     res_sd = f_open(&fnew, GBKCODE_FILE_NAME, FA_OPEN_EXISTING | FA_READ);
52
53     if (res_sd == FR_OK) {
54         f_lseek (&fnew, pos); //指针偏移
55
56         //32*32 大小的汉字 其字模 占用 32*32/8 个字节
57         res_sd = f_read(&fnew, pBuffer, WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8, &br );
58         f_close(&fnew);
59
60         return 0;
61     } else
62         return -1;
63 }
64
65 }
```

当字库的数据源在 SD 卡时, GetGBKCode 宏指向的是这个 GetGBKCode_from_sd 函数。由于字库是使用 SD 卡的文件系统存储的, 从 SD 卡中获取字模数据实质上是直接读取字库文件, 利用 f_lseek 函数偏移文件的读取指针, 使它能够读取特定字符的字模数据。

由于使用文件系统的方式读取数据比较慢, 而 SD 卡大多数都会使用文件系统, 所以我们一般使用 SPI-FLASH 直接存储字库(不带文件系统地使用), 市场上有一些厂商直接生产专用的字库芯片, 可以直接使用, 省去自己制作字库的麻烦。

显示 GB2312 字符示例

下面我们再来看 main 文件是如何利用这些函数显示 GB2312 的字符, 由于我们用 GetGBKCode 宏屏蔽了差异, 所以在上层使用字符串函数时, 不需要针对不同的字库来源写不同的代码, 见代码清单 25-14。

代码清单 28-33 main 函数

```
1
2 int main(void)
3 {
4     ILI9806G_Init ();           //LCD 初始化
5
6
7     /* USART config */
8     USART_Config();
9
10    printf("\r\n ***** 液晶屏中文显示程序 (字模文件在 SD 卡) ***** \r\n");
11    printf("\r\n 实验前请阅读工程中的 readme.txt 文件说明, 存储字模数据到 SPI-FLASH 或 SD 卡\r\n");
12
13
14    //其中 0、3、5、6 模式适合从左至右显示文字,
15    //不推荐使用其它模式显示文字 其它模式显示文字会有镜像效果
16    //其中 6 模式为大部分液晶例程的默认显示方向
17    ILI9806G_GramScan ( 6 );
18
19
20
21    while ( 1 ) {
22        LCD_Test();
23    }
24
25
26 }
```

main 文件中的初始化流程与普通的液晶初始化没有区别, 这里也不需要初始化 SPI 或 SDIO, 因为我们在获取字库的函数中包含了相应的初始化流程。在 while 循环里调用的 LCD_Test 包含了显示 GB2312 字符串的示例, 见代码清单 28-34。

代码清单 28-34 显示 GB2312 字符示例

```
1 /*用于测试各种液晶的函数*/
2 void LCD_Test(void)
3 {
4     /*演示显示变量*/
5     static uint8_t testCNT = 0;
6     char dispBuff[100];
7
8     testCNT++;
9
10    LCD_SetFont(&Font16x32);
11    LCD_SetColors(RED,BLACK);
12
13    ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏, 显示全黑 */
14    /******显示字符串示例******/
15    ILI9806G_DisplayStringLine_EN_CH(LINE(0),"野火 4.5 寸 LCD 参数: ");
16    ILI9806G_DisplayStringLine_EN_CH(LINE(2),"分辨率: 480x854 px");
17    ILI9806G_DisplayStringLine_EN_CH(LINE(3),"ILI9806G 液晶驱动");
18    ILI9806G_DisplayStringLine_EN_CH(LINE(4),"GT5688 触摸屏驱动");
19
20    /******显示变量示例******/
21    LCD_SetTextColor(GREEN);
22
23    /*使用 c 标准库把变量转化成字符串*/
24    sprintf(dispBuff,"显示变量计数 : %d ",testCNT);
25    ILI9806G_ClearLine(LINE(7)); /* 清除单行文字 */
26
27    /*然后显示该字符串即可, 其它变量也是这样处理*/
28    ILI9806G_DisplayStringLine_EN(LINE(7),dispBuff);
29
30    /*以下省略其它液晶演示示例*/
31 }
```

在调用字符串显示函数的时候，我们也是直接使用双引号括起要显示的中文字符即可，为什么这样就能正常显示呢？我们的字符串显示函数需要的输入参数是字符的 GB2312 编码，编译器会自动转化这些中文字符成相应的 GB2312 编码吗？为什么编译器不把它转化成 UTF-8 编码？这跟我们的开发环境配置有关，在 MDK 软件中，可在“Edit->Configuration->Editor->Encoding”选项设定编码，见图 28-40。

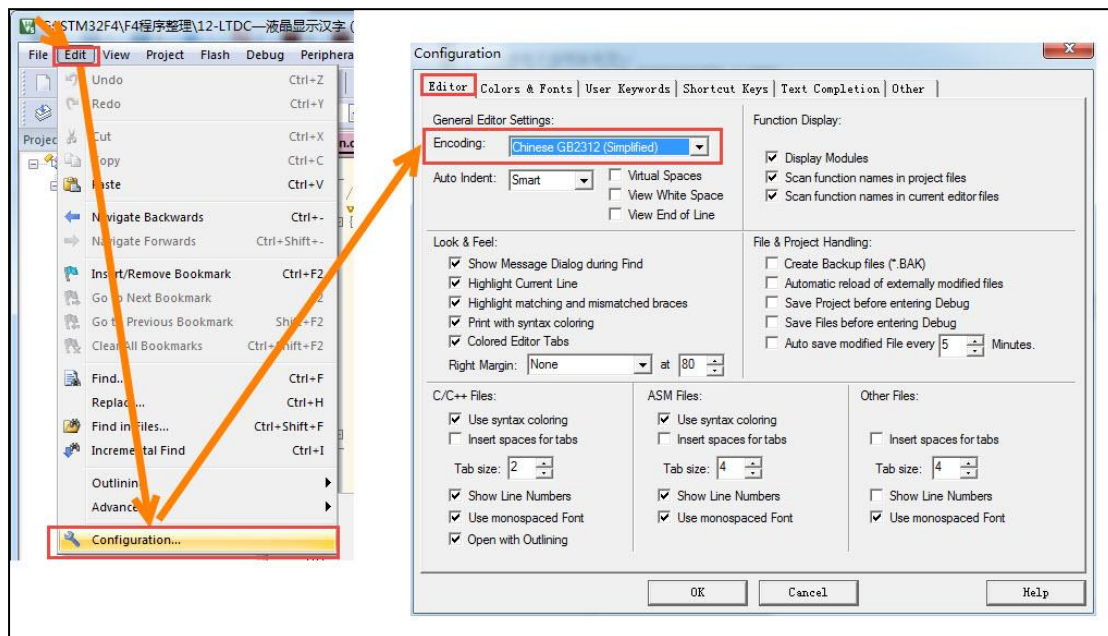


图 28-40 MDK 中的字符编码选项

编译环境会把文件中的字符串转换成这里配置的编码，然后存储到 STM32 的程序空间中，所以这里的设定要与您的字库编码格式一样。如果您的实验板显示的时候出现乱码，请确保以下所有环节都正常：

- ❑ SPI-FLASH 或 SD 卡中是否有字库文件？
- ❑ 文件存储的位置或路径是否与程序的配置一致？
- ❑ 开发环境中的字符编码选项是否与字库的编码一致？

29.3.4 显示任意大小的字符

前文中无论是 ASCII 字符还是 GB2312 的字符，都只能显示字库中设定的字体大小，例如，我们想显示一些像素大小为 48x48 的字符，那我们又得制作相应的字库，非常麻烦。为此我们编写了一些函数，简便地实现显示任意大小字符的目的。本小节的内容请打开“**液晶显示中英文（任意大小）**”工程来配合阅读。

1. 编程要点

- (1) 编写缩放字模数据的函数；
- (2) 编写利用缩放字模的结果进行字符显示的函数；
- (3) 编写测试程序，控制显示不同大小的字符。

2. 代码分析

缩放字模数据

显示任意大小字符的功能，其核心是缩放字模，通过 LCD_zoomChar 函数对原始字模数据进行缩放，见代码清单 28-18。

代码清单 28-35 缩放字模数据

```
1
2 /*****缩放字体*****/
3 #define ZOOMMAXBUFF 16384
4 uint8_t zoomBuff[ZOOMMAXBUFF] = {0}; //用于缩放的缓存，最大支持到 128*128
5 uint8_t zoomTempBuff[1024] = {0};
6
7 /**
8  * @brief 缩放字模，缩放后的字模由 1 个像素点由 8 个数据位来表示
9  *          0x01 表示笔迹，0x00 表示空白区
10  * @param in_width : 原始字符宽度
11  * @param in_heig : 原始字符高度
12  * @param out_width : 缩放后的字符宽度
13  * @param out_heig: 缩放后的字符高度
14  * @param in_ptr : 字库输入指针 注意: 1pixel 1bit
15  * @param out_ptr : 缩放后的字符输出指针 注意: 1pixel 8bit
16  *      out_ptr 实际上没有正常输出，改成了直接输出到全局指针 zoomBuff 中
17  * @param en_cn : 0 为英文，1 为中文
18  * @retval 无
19  */
20 void ILI9806G_zoomChar(uint16_t in_width, //原始字符宽度
21                        uint16_t in_heig, //原始字符高度
22                        uint16_t out_width, //缩放后的字符宽度
23                        uint16_t out_heig, //缩放后的字符高度
24                        uint8_t *in_ptr, //字库输入指针 注意: 1pixel 1bit
25                        uint8_t *out_ptr, //缩放后的字符输出指针 注意: 1pixel 8bit
26                        uint8_t en_cn) //0 为英文，1 为中文
27 {
28     uint8_t *pts,*ots;
29     //根据源字模及目标字模大小，设定运算比例因子，
30     //左移 16 是为了把浮点运算转成定点运算
31     unsigned int xrIntFloat_16=(in_width<<16)/out_width+1;
32     unsigned int yrIntFloat_16=(in_heig<<16)/out_heig+1;
33
34     unsigned int srcy_16=0;
35     unsigned int y,x;
36     uint8_t *pSrcLine;
37
38     uint16_t byteCount,bitCount;
39
40     //检查参数是否合法
41     if (in_width > 48) return; //字库不允许超过 48 像素
42     if (in_width * in_heig == 0) return;
43     if (in_width * in_heig > 48*48 ) return; //限制输入最大 48*48
44
45     if (out_width * out_heig == 0) return;
46     if (out_width * out_heig >= ZOOMMAXBUFF ) return; //限制最大缩放 128*128
47     pts = (uint8_t*)&zoomTempBuff;
48
49     //为方便运算，字库的数据由 1 pixel/1bit 映射到 1pixel/8bit
50     //0x01 表示笔迹，0x00 表示空白区
51     if (en_cn == 0x00) { //英文
```



```
52 //英文和中文字库上下边界不对, 可在此处调整。需要注意 tempBuff 防止溢出
53 for (byteCount=0; byteCount<in_heig*in_width/8; byteCount++) {
54     for (bitCount=0; bitCount<8; bitCount++) {
55         //把源字模数据由位映射到字节
56         //in_ptr 里 bitX 为 1, 则 pts 里整个字节值为 1
57         //in_ptr 里 bitX 为 0, 则 pts 里整个字节值为 0
58         *pts++ = (in_ptr[byteCount] & (0x80>>bitCount)) ? 1:0;
59     }
60 }
61 } else { //中文
62     for (byteCount=0; byteCount<in_heig*in_width/8; byteCount++) {
63         for (bitCount=0; bitCount<8; bitCount++) {
64             //把源字模数据由位映射到字节
65             //in_ptr 里 bitX 为 1, 则 pts 里整个字节值为 1
66             //in_ptr 里 bitX 为 0, 则 pts 里整个字节值为 0
67             *pts++ = (in_ptr[byteCount] & (0x80>>bitCount)) ? 1:0;
68         }
69     }
70 }
71
72 //zoom 过程
73 pts = (uint8_t*)&zoomTempBuff; //映射后的源数据指针
74 ots = (uint8_t*)&zoomBuff; //输出数据的指针
75 for (y=0; y<out_heig; y++) { /*行遍历*/
76     unsigned int srcx_16=0;
77     pSrcLine=pts+in_width*(srcy_16>>16);
78     for (x=0; x<out_width; x++) { /*行内像素遍历*/
79         ots[x]=pSrcLine[srcx_16>>16]; //把源字模数据复制到目标指针中
80         srcx_16+=xrIntFloat_16; //按比例偏移源像素点
81     }
82     srcy_16+=yrIntFloat_16; //按比例偏移源像素点
83     ots+=out_width;
84 }
85 /*!!! 缩放后的字模数据直接存储到全局指针 zoomBuff 里了*/
86 //out_ptr 没有正确传出, 后面调用直接改成了全局变量指针!
87 out_ptr = (uint8_t*)&zoomBuff;
88
89 /*实际中如果使用 out_ptr 不需要下面这一句!!!
90 只是因为 out_ptr 没有使用, 会导致 warning。强迫症*/
91 out_ptr++;
92 }
```

缩放字模的本质是按照缩放比例, 减少或增加矩阵中的像素点, 见图 28-41, 只要把左侧的矩阵隔一行、隔一列地取出像素点, 即可得到右侧按比例缩小了的矩阵, 而右侧的小矩阵按比例填复制像素点即可得到左侧放大的矩阵, 上述函数就是完成了这样的工作。

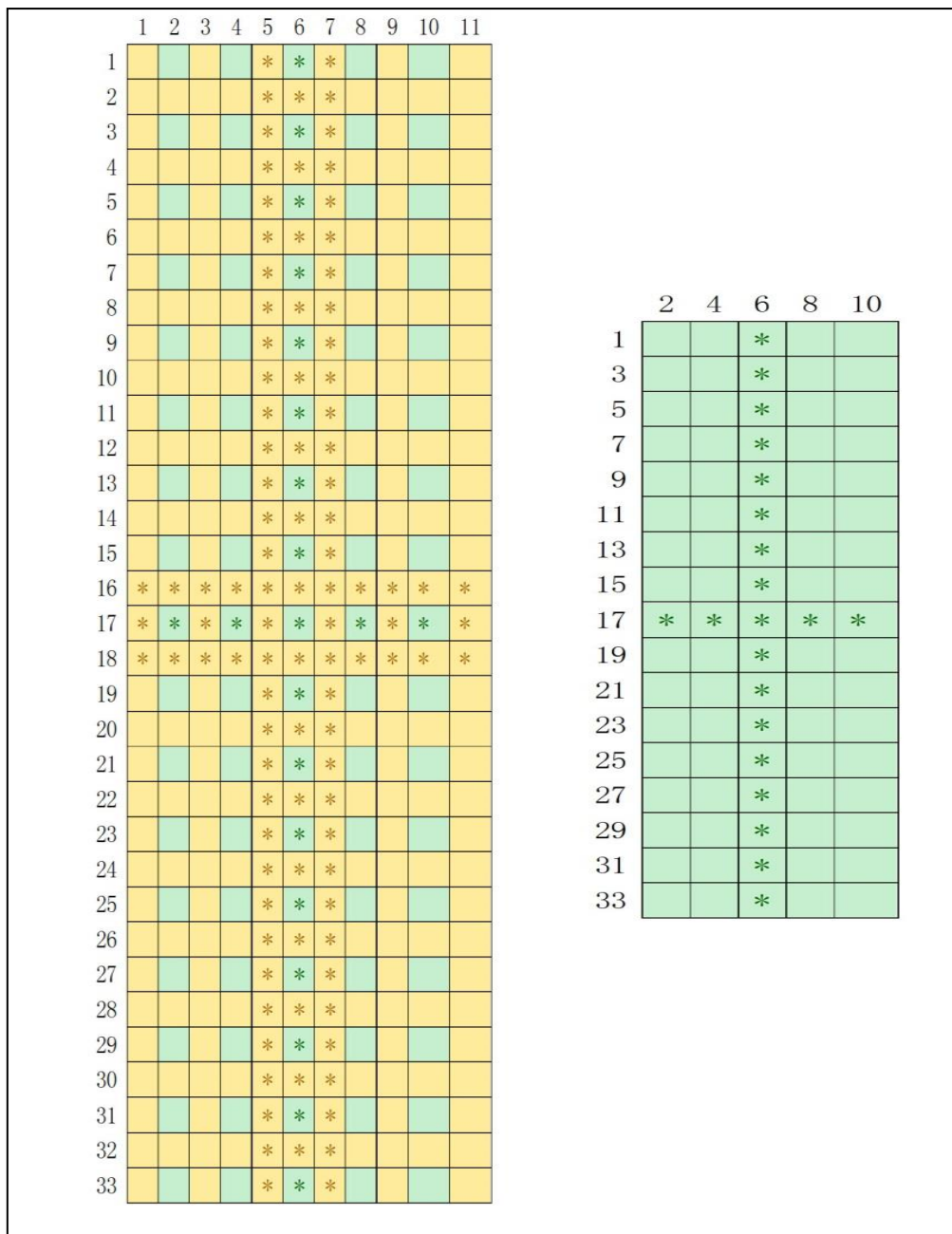


图 28-41 缩放矩阵

该函数的说明如下：

(1) 输入参数

函数包含输入参数源字模、缩放后字模的宽度及高度：in_width、inheig、out_width、out_heig。源字模数据指针 in_ptr，缩放后的字符指针 out_ptr 以及用于指示字模是英文还是中文的标志 en_cn。其中 out_ptr 指针实质上没有用到，这个函数缩放后的数据最后直接存储在全局变量 zoomBuff 中了。

(2) 计算缩放比例

根据输入字模与要求的输出字模大小，计算出缩放比例到 xrIntFloat_16 及 yrIntFloat_16 变量中，运算式中的左移 16 位是典型的把浮点型运算转换成定点运算的

处理方式。理解的时候可把左移 16 位的运算去掉，把它当成一个自然的数学小数运算即可。

(3) 检查输入参数

由于运算变量及数组的一些限制，函数中要检查输入参数的范围，本函数限制最大输出字模的大小为 128*128 像素，输入字模限制不可以超过 48*48 像素。

(4) 映射字模

输入源的字模都是 1 个数据位表示 1 个像素点的，为方便后面的运算，函数把输入字模转化成 1 个字节(8 个数据位)表示 1 个像素点，该字节的值为 0x01 表示笔迹像素，0x00 表示空白像素。把字模数据的 1 个数据位映射为 1 个字节，可以方便后面直接使用指针和数组索引运算。

(5) 缩放字符

缩放字符这部分代码比较难理解，但总的来说它就是利用前面计算得的比例因子，以它为步长复制源字模的数据到目标字模的缓冲区中，具体的抽象运算只能意会了。其中的右移 16 位是把比例因子由定点数转换回原始的数值。如果还是觉得难以理解，可以把函数的宽度及高度输入参数 in_width、inheig、out_width 及 out_heig 都设置成 16，然后代入运算来阅读这段代码。

(6) 缩放结果

经过运算，缩放的结果存储在 zoomBuff 中，它只是存储了一个字模的缩放结果，所以每显示一个字模都需要先调用这个函数更新 zoomBuff 中的字模数据，而且它也是用 1 个字节表示 1 个像素位的。

利用缩放的字模数据显示字符

由于缩放后的字模数据格式与我们原来用的字模数据格式不一样，所以我们要重新编写字符显示函数，见代码清单 28-36。

代码清单 28-36 利用缩放的字模显示字符

```
1
2 /**
3  * @brief 利用缩放后的字模显示字符
4  * @param Xpos : 字符显示位置 x
5  * @param Ypos : 字符显示位置 y
6  * @param Font_width : 字符宽度
7  * @param Font_Heig: 字符高度
8  * @param c : 要显示的字模数据
9  * @param DrawModel : 是否反色显示
10 * @retval 无
11 */
12 void ILI9806G_DrawChar_Ex(uint16_t usX, //字符显示位置 x
13                             uint16_t usY, //字符显示位置 y
14                             uint16_t Font_width, //字符宽度
15                             uint16_t Font_Height, //字符高度
16                             uint8_t *c, //字模数据
17                             uint16_t DrawModel) //是否反色显示
18 {
19     uint32_t index = 0, counter = 0;
20
21     //设置显示窗口
22     ILI9806G_OpenWindow ( usX, usY, Font_width, Font_Height);
```

```
23
24     ILI9806G_Write_Cmd ( CMD_SetPixel );
25
26     //按字节读取字模数据
27     //由于前面直接设置了显示窗口，显示数据会自动换行
28     for ( index = 0; index < Font_Height; index++ ) {
29         //一位一位处理要显示的颜色
30         for ( counter = 0; counter < Font_width; counter++ ) {
31             //缩放后的字模数据，以一个字节表示一个像素位
32             //整个字节值为 1 表示该像素为笔迹
33             //整个字节值为 0 表示该像素为背景
34             if ( *c++ == DrawModel )
35                 ILI9806G_Write_Data ( CurrentBackColor );
36             else
37                 ILI9806G_Write_Data ( CurrentTextColor );
38         }
39     }
40 }
```

注意在这个函数中，它并没有对中英文模区分显示代码，因为本函数的字模是由输入参数 `c` 指针中获取的，在调用本函数时，需要输入要显示的字模数据指针，而不是字符编码。在其它方面这个函数主体与前面介绍的字符显示函数都很类似，只是它在判断字模数据位的时候，直接用一整个字节来判断，区分显示分支，而且还支持了反色显示模式。

利用缩放的字模显示字符串

单个字符显示的函数并不包含字模的获取过程，为便于使用，我们把它直接封装成字符串显示函数，见代码清单 28-37。

代码清单 28-37 利用缩放的字模显示字符串

```
1 /**
2  * @brief  利用缩放后的字模显示字符串
3  * @param  Xpos : 字符显示位置 x
4  * @param  Ypos : 字符显示位置 y
5  * @param  Font_width : 字符宽度，英文字符在此基础上/2。注意为偶数
6  * @param  Font_Heig: 字符高度，注意为偶数
7  * @param  c : 要显示的字符串
8  * @param  DrawModel : 是否反色显示
9  * @retval 无
10 */
11 void ILI9806G_DisplayStringEx(uint16_t x,          //字符显示位置 x
12                               uint16_t y,          //字符显示位置 y
13                               uint16_t Font_width,
14                               //要显示的字体宽度，英文字符在此基础上/2。注意为偶数
15                               uint16_t Font_Height, //要显示的字体高度，注意为偶数
16                               uint8_t *ptr,         //显示的字符内容
17                               uint16_t DrawModel)   //是否反色显示
18 {
19
20
21 {
22     uint16_t Charwidth = Font_width; //默认为 Font_width，英文宽度为中文宽度的一半
23     uint8_t *psr;
24     uint8_t Ascii; //英文
25     uint16_t usCh; //中文
26
27     //占用空间太大，改成全局变量
28     // uint8_t ucBuffer [ WIDTH_CH_CHAR*HEIGHT_CH_CHAR/8 ];
29
30     while ( *ptr != '\0' ) {
```

```
31      /****处理换行*****/
32      if ( ( x - ILI9806G_DispWindow_X_Star + Charwidth ) > LCD_X_LENGTH ) {
33          x = ILI9806G_DispWindow_X_Star;
34          y += Font_Height;
35      }
36
37      if ( ( y - ILI9806G_DispWindow_Y_Star + Font_Height ) > LCD_Y_LENGTH ) {
38          x = ILI9806G_DispWindow_X_Star;
39          y = ILI9806G_DispWindow_Y_Star;
40      }
41
42      if (*ptr > 0x80) { //如果是中文
43          Charwidth = Font_width;
44          usCh = * ( uint16_t * ) ptr;
45          usCh = ( usCh << 8 ) + ( usCh >> 8 );
46          GetGBKCode ( ucBuffer, usCh ); //取字模数据
47          //缩放字模数据, 源字模为 32*32
48          ILI9806G_zoomChar(WIDTH_CH_CHAR,HEIGHT_CH_CHAR,Charwidth,
49                          Font_Height,(uint8_t *)&ucBuffer,psr,1);
50          //显示单个字符
51          ILI9806G_DrawChar_Ex(x,y,Charwidth,Font_Height,
52                              (uint8_t*)&zoomBuff,DrawModel);
53          x+=Charwidth;
54          ptr+=2;
55      } else {
56          Charwidth = Font_width / 2;
57          Ascii = *ptr - 32;
58          //使用 16*32 字体缩放字模数据
59          ILI9806G_zoomChar(16,32,Charwidth,Font_Height,
60          (uint8_t *)&Font16x32.table[Ascii * Font16x32.Height*Font16x32.Width/8],psr,0);
61          //显示单个字符
62          ILI9806G_DrawChar_Ex(x,y,Charwidth,Font_Height,
63                              (uint8_t*)&zoomBuff,DrawModel);
64          x+=Charwidth;
65          ptr++;
66      }
67  }
68 }
```

这个函数包含了从字符编码到源字模获取、字模缩放及单个字符显示的过程，多个这样的过程组合起来，就实现了简单易用的字符串显示函数。而且可了解到它使用的英文源字模数据是 Font16x32 字体，而中文源字模数据仍是采用 GetGBKCode 函数获取，使得数据源的获取与上层分离，支持从 SPI FLASH 及 SD 卡中获取数据源。

利用缩放的字模显示示例

利用缩放的字模显示时，液晶的初始化过程与前面的工程无异，下面我们给出 LCD_Test 函数中调用字符串函数显示不同字符时的示例，见代码清单 28-38。

代码清单 28-38 利用缩放的字模显示示例

```
1  /*用于测试各种液晶的函数*/
2  void LCD_Test(void)
3  {
4      /*演示显示变量*/
5      static uint8_t testCNT = 0;
6      char dispBuff[100];
7
8      testCNT++;
9
10     LCD_SetFont(&Font16x32);
11     LCD_SetColors(RED,BLACK);
12
13     ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); // 清屏，显示全黑 */
```

```
14
15     /*****显示字符串示例*****/
16     ILI9806G_DispStringLine_EN_CH(LINE(0), "野火 BH");
17     //显示指定大小的字符
18     ILI9806G_DisplayStringEx(0, 1*48, 48, 48, (uint8_t *) "野火 BH", 0);
19     ILI9806G_DisplayStringEx(0, 2*56, 56, 56, (uint8_t *) "野火 BH", 0);
20
21     /*****显示变量示例*****/
22     LCD_SetTextColor(GREEN);
23
24     /*使用 c 标准库把变量转化成字符串*/
25     sprintf(dispBuff, "显示变量计数 : %d ", testCNT);
26     ILI9806G_ClearLine(LINE(7)); /* 清除单行文字 */
27
28     /*然后显示该字符串即可，其它变量也是这样处理*/
29     ILI9806G_DispStringLine_EN(LINE(7), dispBuff);
30
31     /*...以下部分省略*/
32 }
```

下载验证

用 USB 线连接开发板，编译程序下载到实验板，并上电复位，各个不同的工程会有不同的的液晶屏显示字符示例。